COMP2017 Practice Exam A

Mock-style paper: files, layout, security, pthreads, parallel search, pipe/select IPC

Question	Topic	Marks
Q1	Simple C / file parsing	10
Q2	Struct and union layout	10
Q3	Security and linking	10
Q4	Concurrency	20
Q5	Parallel pthread programming	20
Q6	Process IPC backend	30
Total		**100**

Instructions

- Answer all questions. Write clear C or pseudocode where allowed.
- Unless a question asks for defensive programming, assume inputs and system calls are valid.
- Where synchronization is required, avoid data races and busy waiting.
- State reasonable assumptions when the question leaves details unspecified.

Question 1 - Simple C / File Parsing (10 marks)

Q1(a) 5 marks. Write a C program that reads a file named **temperatures.csv**. Each line contains one integer temperature. Your program should print the maximum temperature and the average temperature to two decimal places.

```
21
19
25
18
```

Q1(b) 5 marks. Now suppose each line contains a station ID and temperature separated by a comma. Describe how you would modify your code to ignore the station ID and compute the same maximum and average.

```
S101,21
S204,19
S088,25
S101,18
```

Question 2 - Data Structures / Layout (10 marks)

Assume a 64-bit system with normal C alignment. sizeof(char)=1, sizeof(uint8_t)=1, sizeof(uint16_t)=2, sizeof(uint32_t)=4, sizeof(void*)=8.

```
struct packet {
    uint8_t type;
    union {
        struct {
            uint16_t source;
            uint32_t sequence;
        } data;
        struct {
            uint8_t code;
            uint8_t flags;
            uint32_t error_id;
        } error;
    } body;
    char tag;
};

struct packet packets[2];
```

Field	Offset from packets[0]
packets[0].type	
packets[0].body.data.source	
packets[0].body.data.sequence	
packets[0].body.error.code	
packets[0].body.error.flags	
packets[0].body.error.error_id	
packets[0].tag	
packets[1].type	

Question 3 - Security / Linking (10 marks)

Q3(a) 5 marks. Explain the difference between a statically linked library and a dynamically loaded shared object. Mention compile/link time, runtime loading, binary size, and update behaviour.

Q3(b) 5 marks. Identify and explain two security risks introduced by dynamically loaded shared objects. Examples may include library path hijacking, LD_PRELOAD abuse, unexpected version changes, or malicious replacement of a shared object.

Question 4 - Concurrency (20 marks)

A campus print shop accepts print jobs from multiple students and has multiple worker threads printing documents. The current implementation has race conditions and busy waiting.

```
#define MAX_JOBS 16
#define MAX_PRINTERS 4

typedef struct {
    int pages;
    int printer_type;
    int success;
    int complete;
} job_t;
```

```
int paper_stock[MAX_PRINTERS];
job_t *jobs[MAX_JOBS];
int job_head = 0;
int job_tail = 0;
// Declare additional variables as needed.

// You may not modify this function.
void print_job(job_t *job) {
    job->success = 1;
    for (int i = 0; i < job->pages; i++) {
        if (paper_stock[job->printer_type] == 0) {
            job->success = 0;
            goto done;
        }
    }
    paper_stock[job->printer_type] -= job->pages;
done:
    job->complete = 1;
}

void *worker_fn(void *arg) {
    while (1) {
        job_t *job = jobs[job_head];
        job_head = (job_head + 1) % MAX_JOBS;
        print_job(job);
    }
    return NULL;
}

int submit_job(int printer_type, int pages) {
    job_t job = { .printer_type = printer_type, .pages = pages };
    job.complete = job.success = 0;
    jobs[job_tail] = &job;
    job_tail = (job_tail + 1) % MAX_JOBS;
    while (job.complete == 0) ;
    return job.success;
}

void print_shop_init(void) {
    for (int i = 0; i < MAX_PRINTERS; i++) paper_stock[i] = 500;
}
```

Rewrite **worker_fn** and **submit_job**, adding any synchronization required to avoid all race conditions and all busy waiting. You may modify **job_t**, add global variables, and add initialization to **print_shop_init**. Jobs should be printed in parallel where possible when they do not require the same printer type.

Question 5 - Parallel Programming (20 marks)

Q5(a) 15 marks. The following code returns up to **max_offsets** starting offsets at which a word appears in a buffer. Write a parallel version using 10 pthreads. There are no requirements on the order of reported offsets.

```
int find_word(const char *word, const char *buffer,
              int *offsets, int max_offsets) {
    int word_len = strlen(word);
    int buffer_len = strlen(buffer);
    int idx = 0;

    for (int i = 0; i <= buffer_len - word_len; i++) {
        int j;
        for (j = 0; j < word_len; j++) {
            if (buffer[i + j] != word[j]) break;
        }
        if (j == word_len) {
            offsets[idx++] = i;
            if (idx == max_offsets) break;
        }
    }
    return idx;
}
```

Q5(b) 5 marks. Discuss the advantages and disadvantages of using processes created with fork instead of pthreads for this search problem. Your answer should compare shared address spaces, result collection, overhead, and safety isolation.

Question 6 - Process IPC Backend (30 marks)

Write C code for a server backend that manages a fixed number of client processes. The number of clients is passed as the first command line argument. A complete binary named **client** exists in the current working directory.

- The server must create the required number of client processes.

- Each client communicates with the server using two anonymous pipes: one server-to-client pipe and one client-to-server pipe.

- The fd for reading from the server and the fd for writing to the server must be passed as argv[1] and argv[2] to the client binary.

- The only fds open in a client should be stdin, stdout, stderr, and the two pipe fds used for communication.
- When the server receives a message from one client, it should broadcast that message to all other clients.
- The server terminates after all client-to-server pipes have been closed by the clients.
- If the server receives SIGTERM, it must send SIGTERM to all live clients before terminating.

You may use select, read, write, pipe, fork, execv, signal or sigaction, kill, and waitpid.

Examiner Blueprint

This page is not a solution guide. It records the intended assessment focus behind the paper.

Question	Intended signal	Core skills tested
Q1	Small file program	fopen/fgets or fscanf, integer accumulation, count, max, average, CSV parsing with sscanf.
Q2	Layout calculation	Padding, alignment, union members starting at offset 0 within the union, array stride via sizeof(struct).
Q3	Security concept	Static vs dynamic linking, library load paths, runtime replacement, ABI/version risk.
Q4	Producer-consumer	Bounded circular queue, mutex/condvar, per-job completion condition, resource-specific locks.
Q5	Parallel pthreads	Fixed partition of possible starting offsets, shared counter with mutex, join threads, fork vs thread discussion.
Q6	IPC synthesis	fork/exec/pipe fd discipline, select readiness, broadcast loop, EOF handling, SIGTERM cleanup.